61.44 MS/s with zero sample loss

Real-time spectrum analysis with a USRP B210 clone, CyberEther and a laptop GPU. A practical methodology for low-budget R&D.

Result	Live spectrum observation
Sample rate	61.44 MS/s complex (the AD9361 maximum)
Throughput	491 MB/s of CF32 samples, steady
Buffer fill	about 13 %
Buffer loss / radio overflows	0 % / 0 OVF over a 15-minute run
Hardware	Dell Precision 3560 laptop (i7-1185G7, Iris Xe, NVIDIA T500 2 GB), B210 clone on USB 3

A software-defined radio, an open-source analysis program and an ordinary engineering laptop are enough to observe 61.44 MHz of spectrum live without dropping a single sample. Getting there was not obvious: the default settings lost between 2 % and 60 % of the samples. This document records each bottleneck, how it was found (profiling, not guessing), the fix, and the measured result, so that others can reproduce it on similar hardware.

1. The pipeline

Stage	Runs on	Data
B210 clone (AD9361 + FPGA image)	radio	sc16 over USB 3: 61.44 MS/s × 4 B = 245.76 MB/s
UHD 4.9 via SoapySDR / SoapyUHD	CPU (2 threads)	converted to CF32: 491.52 MB/s
CyberEther SoapySDR block	CPU	ring buffer, batches of FFT frames
Duplicate block (to CUDA)	CPU → GPU	pinned host memory, read by the GPU over PCIe
Spectrum analyzer (window, FFT, dB, scaling)	NVIDIA T500 (CUDA)	line plot + waterfall
Display (Vulkan)	Intel Iris Xe	GUI rendering

The key idea is to give each chip the job it is best at: the CPU feeds the USB link, the discrete GPU does the heavy math, and the integrated GPU, which already drives the laptop screen, draws the result.

2. The measurement journey

Step	Configuration	Fill	Loss	Throughput
0	Stock build: rates above ~4 MS/s refused (see 3.1)	low	0 %	far below 61.44 MS/s
1	Rate fix, debug-optimized build, CPU, Batches 32 / Mult. 32	98 %	10 %	—
2	CUDA, Duplicate with managed memory	99 %	60 %	—
3	Debug-optimized build, CPU, Batches 8 / Mult. 4	18–82 %	≈ 3.6 %	450–491 MB/s
4	Release + AVX-512 build, CPU, Batches 8	20–90 %	11 %	420–491 MB/s
5	Release + AVX-512 build, CPU, Batches 16	—	≈ 2 %	470–491 MB/s
6	CUDA, Duplicate with pinned memory, 256 USB frames	13 %	0 %	491 MB/s, 1 OVF burst in 8 min

Step	Configuration	Fill	Loss	Throughput
7	Same, 1024 USB frames	13 %	0 %	491 MB/s, 0 OVF in 15 min

Two lessons: an early 0 % reading was misleading, most likely because the radio was not yet running at the requested rate; and bigger buffers (step 1) do not help when the consumer is simply too slow, they only delay the overflow.

3. The fixes, one by one

3.1 Let the sample rate through (CyberEther patch 1)

Symptom. Rates above about 4 MS/s were rejected as "not supported"; the radio stayed at a low rate, which most likely explains an early 0 % loss reading.

Cause. UHD reports the B200-series rates as a discrete list master_clock / n for the current master clock, but it reconfigures the clock automatically when another rate is requested. CyberEther validated the request against that list and refused anything outside it.

Fix. Only reject non-finite or non-positive rates; warn, but still request, rates outside the reported ranges, and let the driver decide. A unit test covers it. Verified on the B210: 5 MS/s → 40 MHz clock; 61.44 MS/s → 61.44 MHz; 30.72 MS/s → 30.72 MHz. The diff is in Appendix A; it will be proposed upstream.

3.2 Use a known-good FPGA image

The clone is used with an FPGA bitstream that had already been tested without lost packets. UHD is pointed at it with `UHD_IMAGES_DIR` and the `fpga=` device argument:

```
export UHD_IMAGES_DIR=$HOME/uhd_images
driver=uhd,type=b200,fpga=/home/USER/uhd_images/usrp_b210_fpga_github.bin,num_recv_frames=1024
```

3.3 Build an optimized binary (release, AVX-512)

Profiling with `perf` showed one thread at 95 % of a core doing the entire spectrum chain, while 65 % of the machine was idle. The default debugoptimized build (-O2, generic x86-64) spent time in `__mulsc3` (the slow, NaN-safe complex multiply) and used only 4-wide SSE vectors.

```
meson setup build-release -Dbuildtype=release -Dnative=true \
    -Dexamples=true -Dtests=false -Dc_std=gnu17 -Dcpp_args=-fcx-limited-range
```

- `-Dnative=true` adds `-march=native` (AVX2/AVX-512 on Tiger Lake).
- `-fcx-limited-range` replaces `__mulsc3` with a direct complex multiply; it only changes results for infinities/NaNs, which do not occur in radio samples.
- `-Dc_std=gnu17` is needed because the bundled LimeSuite does not build as C23 with GCC 15.

3.4 The non-obvious one: Batches must be a multiple of 16

After the optimized build, loss increased from 3.6 % to 11 %. The profile showed the FFT share jump from 12 % to 27 %, and the FFT kernel had changed from `cmplx<float vector(4)>` to plain `cmplx<float>`.

pocketfft vectorizes across transforms: it processes groups of VLEN FFTs at once, and the remainder one by one. VLEN for float is 4 with SSE, 8 with AVX and **16 with AVX-512**. With 8 batches there is no complete group of 16, so every FFT ran scalar. Setting **Batches = 16** (or 32) restored full-width vectorization and brought loss down to about 2 %. Rule of thumb: Batches = a multiple of the SIMD width in floats of the build you run.

3.5 GPU for compute, Intel for display

CyberEther computes on the CPU or on CUDA; Vulkan is used for rendering. `--device-index` selects both the Vulkan and the CUDA device. With PRIME in on-demand mode, the Vulkan device list is 0 = Intel Iris Xe, 1 = NVIDIA T500, while the T500 is the only CUDA device (index 0). So the

default index 0 gives exactly the split we want: **display on the Intel iGPU, math on the NVIDIA dGPU**, and the dGPU does not waste time on the GUI.

The SoapySDR block only produces CPU tensors and CUDA modules refuse CPU inputs, so a **Duplicate** block with `outputDevice: cuda` and `hostAccessible: true` bridges the two (flowgraph in section 4).

3.6 Pinned memory instead of managed memory (CyberEther patch 2)

Symptom. With the GPU path, loss went up to 60 %, with the GPU only 40 % busy.

Cause. Host-accessible CUDA buffers were allocated with `cudaMallocManaged`. The CPU writes each frame, the GPU reads it, and the pages migrate back and forth every frame, which is slow on a Turing GPU.

Fix. Allocate them with `cudaHostAlloc(..., cudaHostAllocMapped | cudaHostAllocPortable)`: the CPU writes to pinned RAM at full speed and the GPU reads it directly over PCIe (Gen4 x4 here; 491 MB/s is a small fraction of it). Result: fill 13 %, 0 % loss, GPU about 36 % busy. The diff is in Appendix B. It changes every host-accessible CUDA buffer, so it should be reviewed upstream before being generalized.

3.7 Deeper USB receive queue

With everything else fixed, one burst of overflows ("O" printed by UHD) appeared in 8 minutes: a short system hiccup during which the host did not read the USB link. With `num_recv_frames=256` the queue holds about 8 ms of samples; `num_recv_frames=1024` raises it to about 34 ms for only ~8 MB of RAM. After that: 0 OVF over 15 minutes.

4. Reproducing it

Flowgraph (CyberEther v1.19, YAML version 2), trimmed to the relevant fields:

```
graph:
  - name: soapy
    module: soapy
    device: cpu
    config:
      deviceString: 'driver=uhd,type=b200,fpga=/home/USER/uhd_images/usrp_b210_fpga_github.bin,num_recv_frames=1024'
      antenna: 'RX2'
      frequency: 94900000
      sampleRate: 61440000
      numberOfBatches: 16        # multiple of 16 on an AVX-512 build
      numberOfTimeSamples: 8192  # FFT size
      bufferMultiplier: 4
  - name: to_gpu
    module: duplicate
    device: cpu
    config: { outputDevice: cuda, hostAccessible: true }
    input:  { buffer: '${graph.soapy.output.signal}' }
  - name: spectrum_analyzer
    module: spectrum_analyzer
    device: cuda
    input:  { buffer: '${graph.to_gpu.output.buffer}' }
```

Launch (a two-letter wrapper script does this and writes the log to a file):

```
UHD_IMAGES_DIR=$HOME/uhd_images stdbuf -oL -eL cyberether -v b210.yml > last.log 2>&1
```

Practical notes from the setup:

- **Build memory.** A parallel build froze a 16 GB laptop with no swap. Use `-j 4`, a cgroup cap (`systemd-run --user --scope -p MemoryMax=10G`) and zram swap.
- **CUDA on Ubuntu.** Meson expects the NVIDIA toolkit layout; with Ubuntu's `nvidia-cuda-toolkit`, a small directory of symlinks (bin, version.txt, targets/x86_64-linux/{include,lib}) and `CUDA_PATH` pointing at it is enough.

- **Live / persistent USB Ubuntu.** `cuInit` returned 999 until `nvidia-uvm` was loaded and its device node created; a udev rule makes it permanent.
- **Measure, don't guess.** `perf record -g -p PID` and `top -H` found every real bottleneck; several plausible guesses (CPU governor, sample format) were wrong.
- **Watch the right counters.** CyberEther's Buffer Loss is cumulative since start (it decays after a startup glitch); UHD's "O" means the radio itself overflowed. Throughput should read 491 MB/s at 61.44 MS/s.

5. Caveats

- 61.44 MS/s exceeds the AD9361's ~56 MHz analog bandwidth (UHD warns about it): the band edges are attenuated.
- Results are for this clone, this FPGA image and this laptop; zero loss was verified over 15 minutes, not days.
- Patch 2 is a local experiment; patch 1 is being proposed upstream. Both apply to CyberEther commit fa027fb8.
- The CPU-only path still loses about 2 % at 61.44 MS/s because the spectrum chain runs on a single thread; the GPU path removes that limit.

6. What's next

What was demonstrated here is **live spectrum observation** (line plot and waterfall) at the full 61.44 MS/s rate. The spectrum is only the beginning. The same lossless pipeline opens the door to:

- **Capture** of raw IQ to a fast SSD (245.76 MB/s in sc16) for offline analysis and reproducible experiments.
- **Real-time processing** of the stream with custom algorithms on the GPU: detection, demodulation, classification, filtering.
- **Replay** of captures through the same flowgraphs to develop algorithms without the radio.

All of this with open-source software and hardware that costs a fraction of a laboratory instrument: a real step towards democratizing signal analysis for low-budget R&D projects.

7. Acknowledgements

- **@ossdrlab**, maker of the **USRP B210 clone**, for affordable, capable hardware.
- **@JamesMcPherson**, for the **FPGA image** that runs the clone without lost packets.
- **@luigifcruz** and the **CyberEther** contributors (github.com/luigifcruz/CyberEther), for the fast, modern, open-source software that computes and displays the spectrum.
- Ettus Research / NI for **UHD**, and the Pothos project for **SoapySDR**.
- **Claude Code** (Anthropic), which did the installation, profiling, patches and this write-up together with the author.

Appendix A — Patch 1: accept sample rates outside the reported ranges

src/domains/io/soapy/library/soapysdr.cc (a unit test is added in module_tests.cc).

```
diff --git a/src/domains/io/soapy/library/soapysdr.cc b/src/domains/io/soapy/library/soapysdr.cc
index 0c913f5c..60d26a40 100644
--- a/src/domains/io/soapy/library/soapysdr.cc
+++ b/src/domains/io/soapy/library/soapysdr.cc
@@ -317,7 +317,10 @@ void SoapyReceiver::queryGainRange() {
 }

 Result SoapyReceiver::validateSettings(const F32 sampleRate, const F32 frequency) const {
-    if (!SampleRateInRanges(sampleRateRanges, sampleRate)) {
+    // The reported sample rate ranges are only checked when the rate is applied. Some
+    // drivers (e.g. UHD) report the rates reachable with the current master clock but
+    // reconfigure the clock on demand, so rates outside the ranges are still valid.
+    if (!std::isfinite(sampleRate) || sampleRate <= 0.0f) {
         JST_ERROR("[MODULE_SOAPY] Sample rate ({:.3f} MHz) not supported.", sampleRate / 1e6);
         return Result::ERROR;
     }
@@ -406,11 +409,16 @@ Result SoapyReceiver::setSampleRate(const F32 sampleRate) {
         return Result::ERROR;
     }

-    if (!SampleRateInRanges(sampleRateRanges, sampleRate)) {
+    if (!std::isfinite(sampleRate) || sampleRate <= 0.0f) {
         JST_WARN("[MODULE_SOAPY] Sample rate ({:.3f} MHz) not supported.", sampleRate / 1e6);
         return Result::WARNING;
     }

+    if (!SampleRateInRanges(sampleRateRanges, sampleRate)) {
+        JST_WARN("[MODULE_SOAPY] Sample rate ({:.3f} MHz) is outside the ranges reported "
+                 "by the device. Requesting it anyway.", sampleRate / 1e6);
+    }
+
     if (library->api().setSampleRate(device, SoapyAbi::RX, 0, sampleRate) != 0) {
         JST_ERROR("[MODULE_SOAPY] Failed to set sample rate: {}", failure());
         return Result::ERROR;
```

Appendix B — Patch 2: pinned host memory for host-accessible CUDA buffers

src/memory/buffer_cuda.cc

```
diff --git a/src/memory/buffer_cuda.cc b/src/memory/buffer_cuda.cc
index 7d5c3a0d..39e33c6e 100644
--- a/src/memory/buffer_cuda.cc
+++ b/src/memory/buffer_cuda.cc
@@ -51,10 +51,12 @@ class CudaBackend final : public CudaBufferBackend, public Backend {
                 JST_ERROR("[MEMORY:BUFFER:CUDA] Allocation size {} is too large.", bytes);
                 return Result::ERROR;
             }
-            JST_CUDA_CHECK(cudaMallocManaged(&buffer, allocBytes), [&] {
-                JST_ERROR("[MEMORY:BUFFER:CUDA] Failed to allocate managed memory: {}.", err);
+            // Mapped pinned host memory: the CPU writes at full speed and kernels read it
+            // over PCIe, without the per-frame page migrations of managed memory.
+            JST_CUDA_CHECK(cudaHostAlloc(&buffer, allocBytes, cudaHostAllocMapped | cudaHostAllocPortable), [&] {
+                JST_ERROR("[MEMORY:BUFFER:CUDA] Failed to allocate pinned host memory: {}.", err);
             });
-            allocationKind = AllocationKind::Managed;
+            allocationKind = AllocationKind::Pinned;
             hostAccessibleFlag = true;
             locationState = Location::Unified;
 #if !defined(JST_OS_WINDOWS)
@@ -340,7 +342,8 @@ class CudaBackend final : public CudaBufferBackend, public Backend {
     bool deviceNative() const override {
         return allocationKind == AllocationKind::Device ||
                allocationKind == AllocationKind::VirtualMemory ||
-               allocationKind == AllocationKind::Managed;
+               allocationKind == AllocationKind::Managed ||
+               allocationKind == AllocationKind::Pinned;
     }
```

```
     bool exportableDeviceMemory() const override {
@@ -384,6 +387,9 @@ class CudaBackend final : public CudaBufferBackend, public Backend {
                 case AllocationKind::Device:
                     cudaFree(buffer);
                     break;
+                case AllocationKind::Pinned:
+                    cudaFreeHost(buffer);
+                    break;
                 default:
                     break;
             }
@@ -407,6 +413,7 @@ class CudaBackend final : public CudaBufferBackend, public Backend {
         None,
         Device,
         Managed,
+        Pinned,
         VirtualMemory,
         ImportedHost,
         ImportedVulkan,
```